

Code-Layout, Readability and Reusability

Date	02 July 2026
Team ID	
Project Name	Personalized Networking Assistant
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes/No/Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code.	Yes	Standard Python 4-space indentations.
2	Proper File Structure	Files and folders are logically organized.	Yes	Split into schemas, routers, services, main.
3	Meaningful Variable Names	Variables reflect their purpose clearly.	Yes	Explicit names like HISTORY_FILE.
4	Function / Method Names	Functions are descriptively named.	Yes	E.g., extract_event_themes().
5	Code Comments	Inline and block comments explain logic.	Yes	Docstrings present on all files.
6	Modular Design	Code is split into reusable functions/modules.	Yes	Seeding and models cached at startup.
7	No Redundant Code	Duplicate or unused code is removed.	Yes	Tested via python linting check.
8	Error Handling	Exceptions and errors are handled gracefully.	Yes	Catches wiki failures and JSON errors.

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High/Medium/Low)
1	event_analyzer classifier	Python / transformers pipeline	Reused inside standalone test validations and routing logic.	High
2	history_logger file handlers	Python / pathlib	Reused for session history logging and admin analytics.	High

3	fact_checker summary API wrapper	Python / requests	Reused for both quick fact-check query route and test cases.	Medium
---	----------------------------------	-------------------	--	--------

Modularity and Python Clean Code Guidelines Followed:

The codebase strictly adheres to the PEP 8 styling guide for Python. Variables and function names are written in clean `snake\_case`, and all service files feature comprehensive docstrings and inline comments detailing parameters. We implement type hinting on all schemas and signatures, allowing static linters to verify correctness. The separation of concerns ensures that code blocks do not exceed 50 lines, and database files are isolated from model loading contexts. This modularity facilitates easy testing, fast code audits, and reuse in mobile app integrations.